

Infrastructure as Code (IaC) & Configuration Management

Terraform

- **Infrastructure as Code (IaC)**

Infrastructure as Code (IaC) is the practice of managing and provisioning infrastructure using machine-readable configuration files instead of manually configuring resources.

With IaC, infrastructure can be:

1. Version controlled
2. Automated
3. Easily modified or destroyed

- **Terraform**

Terraform is an Infrastructure as Code tool developed by HashiCorp. It allows infrastructure and resources to be defined using declarative configuration files. This configuration files are written in **HashiCorp Configuration Language (HCL)**.

The basic workflow terraform follows:

main.tf -> terraform init -> terraform plan -> terraform apply -> Resources created -> terraform destroy -> Resources removed

- **Terraform State**

Terraform maintains a **state file** to keep track of the resources it manages.

Terraform uses state to determine:

1. Which resources it manages
2. The current known state of those resources
3. What changes are required during the next terraform plan

The default local state file is: **terraform.tfstate**

- **Variables and Outputs**

Variables allow Terraform configurations to accept reusable values instead of hard-coding them.

Variables were used for values such as: Project name, Environment, Output filename

Outputs display useful information after Terraform operations.

Variable example:

```
variable "project_name" {  
  description = "Name of the project"  
  type        = string  
  default     = "DevOps Internship"  
}
```

Output example:

```
output "created_file" {
  description = "Path of the file created by Terraform"
  value      = local_file.project_info.filename
}
```

- **Modules**

A **Terraform module** is a set of Terraform configuration files in a single directory. Even a simple configuration consisting of a single directory with one or more .tf files is a module.

Modules are particularly useful for larger infrastructure projects where the same infrastructure pattern needs to be deployed multiple times.

Ansible

- **Ansible**

Ansible is an **open-source IT automation tool** primarily used for **configuration management, application deployment, and orchestration**.

Unlike Terraform, which primarily focuses on provisioning infrastructure, Ansible is commonly used to **configure already-existing machines**.

It allows you to define the desired state of systems in simple, human-readable YAML files called **playbooks**.

- **Inventory**

The Inventory is a file (usually located at /etc/ansible/hosts) that lists all the Managed Nodes.

Static Inventory: A simple text file listing IP addresses or hostnames.

Dynamic Inventory: Scripts that dynamically query cloud providers (AWS, Azure, GCP) to fetch the list of currently running instances.

- **Playbooks**

An Ansible Playbook is a file containing the tasks Ansible should execute.

Written in YAML, they define a list of tasks to be executed across a set of hosts.

The playbook created in this activity:

1. Installed Apache.
2. Created a dev user.
3. Started and enabled the Apache service.

Executed using: `ansible-playbook -i inventory playbook.yml`

- **Terraform vs Ansible**

Feature	Terraform	Ansible
Type	Infrastructure as Code (IaC) tool used to provision and manage cloud infrastructure resources.	Configuration management tool used to automate tasks like software installation, updates, and system configuration.
Language/Syntax	HCL, Declarative	YAML, Procedure
Execution Method	Builds an execution plan before applying changes.	Executes tasks sequentially as defined in playbooks.
State Management	Maintains a state file to track current infrastructure.	Stateless by default.
Agent requirement	No agents required; communicates with cloud provider APIs.	Agentless, uses SSH to communicate with managed nodes, requiring minimal setup.

Name: Purva Wankhede

Batch: B1

Date: 19 Aug 2026